TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos

Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: Evelyn Jasmin Romero Muñoz.

PARALELO: "A".

SEMESTRE: Quinto

ACTIVIDAD #1: Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2026 - ENERO 2027

Actividades a desarrollar en la Unidad #1

Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

1. INTRODUCCIÓN

El presente trabajo tiene como finalidad diseñar la arquitectura lógica de un Sistema de Control de Rescate y Adopción Animal. Este sistema busca facilitar el registro de animales que necesitan ayuda, la asignación de voluntarios y veterinarios, y el seguimiento de los procesos de rescate y adopción.

Para realizar el diseño se identificarán los actores y requerimientos del sistema, se elaborarán diagramas UML y se aplicarán patrones de diseño que permitan organizar mejor sus componentes. También se utilizará GitHub con el flujo de trabajo Gitflow para registrar los avances y mantener un control ordenado de las versiones del proyecto.

2. OBJETIVOS

2.1. Objetivo general

Diseñar la arquitectura lógica de un Sistema de Control de Rescate y Adopción Animal mediante el uso de diagramas UML, patrones de diseño y herramientas de control de versiones, con el propósito de establecer una solución organizada, mantenible y escalable.

2.2. Objetivos específicos

- Identificar los actores y requerimientos funcionales y no funcionales del sistema.
- Representar la estructura y el funcionamiento del sistema mediante diagramas UML.
- Aplicar patrones de diseño que faciliten la organización y el mantenimiento del software.
- Establecer un flujo de trabajo Gitflow en GitHub para registrar los avances del diseño.

3. ANÁLISIS DEL PROBLEMA

El rescate y la adopción de animales son actividades importantes para ayudar a los animales que se encuentran abandonados, perdidos o en peligro. Sin embargo, cuando la información se lleva de manera desorganizada, puede ser difícil conocer los casos registrados, saber quién se encargará de atenderlos y realizar un seguimiento adecuado.

Por esta razón, se propone diseñar un Sistema de Control de Rescate y Adopción Animal que permita registrar los reportes, identificar la especie del animal, asignar voluntarios o veterinarios y mantener actualizada la información de cada caso. Además, el sistema permitirá organizar las solicitudes de adopción y facilitar la comunicación entre las personas responsables.

3.1. Identificación de los actores

Los actores son las personas que interactúan con el Sistema de Control de Rescate y Adopción Animal. Cada uno tiene funciones específicas que permiten registrar, atender y dar seguimiento a los casos de rescate y adopción.

Actor	Función dentro del sistema
Ciudadano o reportante	Registra los animales que necesitan rescate.
Voluntario	Atiende los rescates y actualiza el estado del caso.
Veterinario	Registra la valoración y la atención del animal.
Administrador del refugio	Asigna responsables y supervisa los casos.
Adoptante	Consulta animales disponibles y solicita una adopción.

4. REQUERIMIENTOS DEL SISTEMA

Los requerimientos del sistema describen las funciones que debe cumplir el Sistema de Control de Rescate y Adopción Animal para facilitar el registro y seguimiento de los animales rescatados y ayudar a encontrarles un hogar responsable.

4.1. Requerimientos funcionales

RF01. Registro de rescates: El sistema permitirá registrar un caso de rescate con la ubicación, descripción y fecha del reporte.

RF02. Clasificación de animales: Permitirá clasificar los animales según su especie, como perro, gato o fauna silvestre.

RF03. Asignación de responsables: El administrador podrá asignar un voluntario o veterinario para atender cada caso.

RF04. Seguimiento del rescate: Los responsables podrán actualizar el estado del caso, por ejemplo: reportado, asignado, en atención y resuelto.

RF05. Notificaciones: El sistema enviará notificaciones a los voluntarios cuando se registre un nuevo caso o se les asigne una atención.

RF06. Gestión de adopciones: Permitirá registrar animales disponibles para adopción y recibir solicitudes de personas interesadas.
RF07. Registro de atención: El veterinario podrá registrar las valoraciones y observaciones correspondientes a la atención del animal.

4.2. Requerimientos no funcionales

RNF01. Escalabilidad: La arquitectura deberá permitir agregar nuevas funciones y atender un mayor número de casos.
RNF02. Mantenibilidad: El sistema deberá organizarse en componentes independientes para facilitar las modificaciones.
RNF03. Seguridad: El acceso a las funciones administrativas y a los datos deberá controlarse mediante permisos.
RNF04. Trazabilidad: Los cambios del diseño y del proyecto deberán registrarse mediante Git y GitHub.
RNF05. Usabilidad: La plataforma deberá presentar opciones claras y fáciles de comprender para los usuarios.

5. SOLUCIÓN PROPUESTA

La solución consiste en diseñar un sistema organizado por componentes, que permita administrar los casos de rescate y adopción de animales. Para lograrlo, se propone utilizar el patrón MVC para separar la interfaz, la lógica de control y los datos; Factory Method para crear expedientes según la especie; Observer para gestionar las notificaciones y Singleton para centralizar el acceso al gestor de casos.
Asimismo, se utilizará GitHub con el modelo Gitflow para organizar las versiones del proyecto, registrar los avances y mantener un historial de los cambios realizados.

6. GitHub y metodología Gitflow

Gitflow es un modelo de trabajo que permite organizar el desarrollo de un proyecto mediante ramas. Su propósito es separar los avances en desarrollo de las versiones que se consideran estables, facilitando el control de cambios y la colaboración (Regalado, 2025).
En el Sistema de Control de Rescate y Adopción Animal se propone utilizar las siguientes ramas:

- master o main: contiene la versión estable del proyecto.
- develop: integra los avances que se encuentran en desarrollo.
- feature/analisis-problema: permite trabajar en el análisis de los requerimientos.

- feature/diagramas-uml: se utiliza para organizar los diagramas del sistema.
- feature/patrones-diseno: permite documentar los patrones de diseño seleccionados.
- feature/documentacion: se utiliza para organizar y actualizar el informe.

Los cambios de cada rama de trabajo se integrarán en develop después de su revisión. Cuando el proyecto esté listo, se podrá integrar la versión final en la rama principal.

El uso de GitHub permite mantener un historial de versiones, identificar los cambios realizados y organizar el trabajo de manera clara. Esto favorece la trazabilidad y el mantenimiento del diseño.

Enlace del repositorio: [https://github.com/eromero2284/sistema-rescate-adopcion-animal].

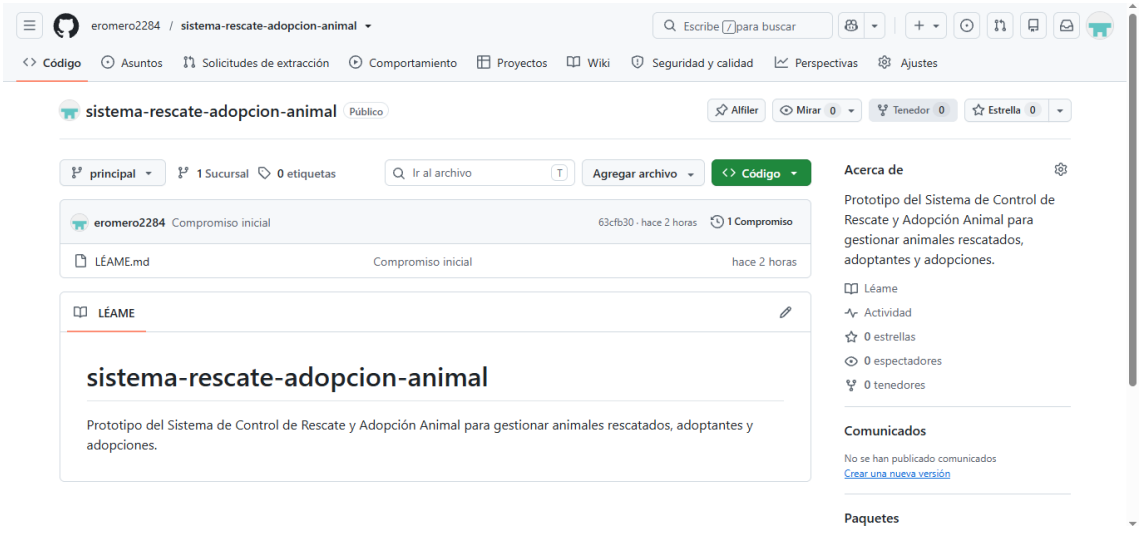


Figura 1. *Repositorio creado*

Fuente: *github.com*

7. DISEÑO DEL SISTEMA

7.1. Arquitectura general

El Sistema de Control de Rescate y Adopción Animal se organizará en componentes que permitan separar las funciones de la interfaz, el procesamiento de las solicitudes y la administración de los datos.

La arquitectura propuesta estará formada por las siguientes capas:

1. **Capa de presentación:** permite que el ciudadano, el voluntario, el veterinario y el administrador interactúen con el sistema mediante pantallas y formularios.
2. **Capa de control:** recibe las acciones realizadas por los usuarios y coordina las operaciones necesarias.
3. **Capa de lógica de negocio:** contiene las reglas para registrar rescates, clasificar especies, asignar responsables y actualizar estados.
4. **Capa de acceso a datos:** administra el almacenamiento y la consulta de los expedientes.
5. **Sistema de notificaciones:** comunica al personal responsable la existencia de nuevos casos o actualizaciones importantes.

El patrón MVC se utilizará para organizar la interacción entre las vistas, los controladores y los modelos. Los patrones Singleton, Observer y Factory Method complementarán el diseño para mejorar la organización y reutilización de los componentes.

El repositorio de GitHub y las ramas Gitflow se utilizarán para gestionar las versiones del diseño y registrar sus avances.

7.2. Diagrama de arquitectura

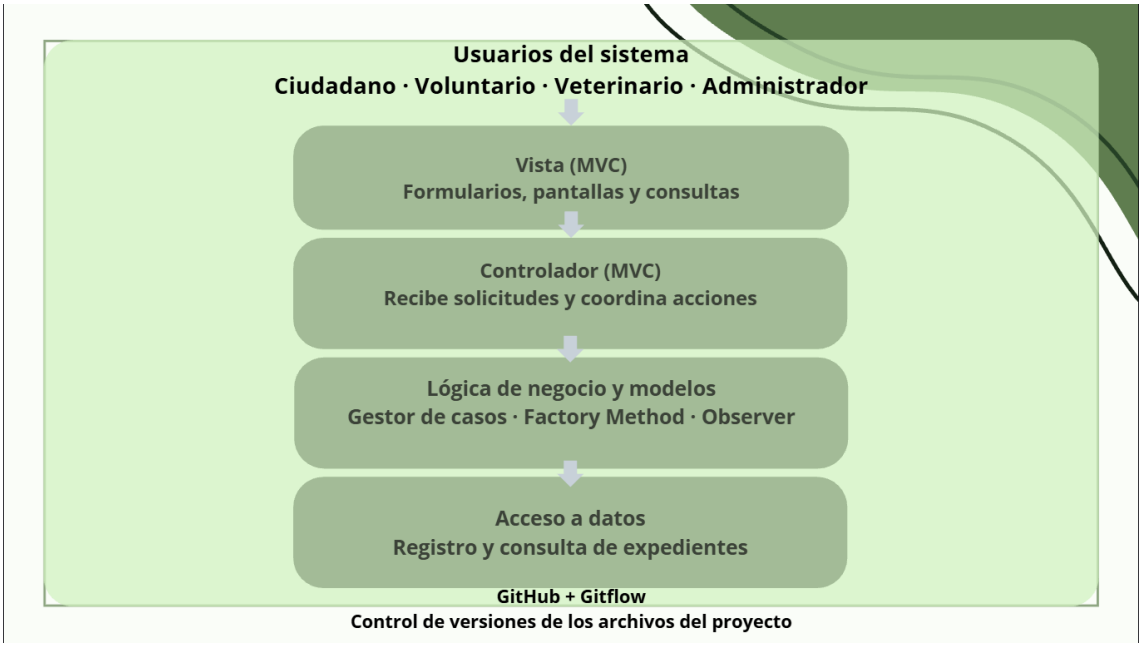


Figura 2: *diseño de diagrama de arquitectura*

Fuente: *Elaboración propia.*

7.3. Diagrama de casos de uso

El diagrama de casos de uso representa las principales interacciones entre los usuarios y el Sistema de Control de Rescate y Adopción Animal.

El ciudadano puede reportar un rescate y consultar información de los casos. El voluntario y el veterinario pueden revisar los casos asignados y actualizar su estado de acuerdo con sus funciones. El administrador puede consultar los registros, asignar responsables y gestionar las adopciones.

Este diagrama permite identificar las funciones principales del sistema y las responsabilidades de cada actor antes de iniciar su implementación.

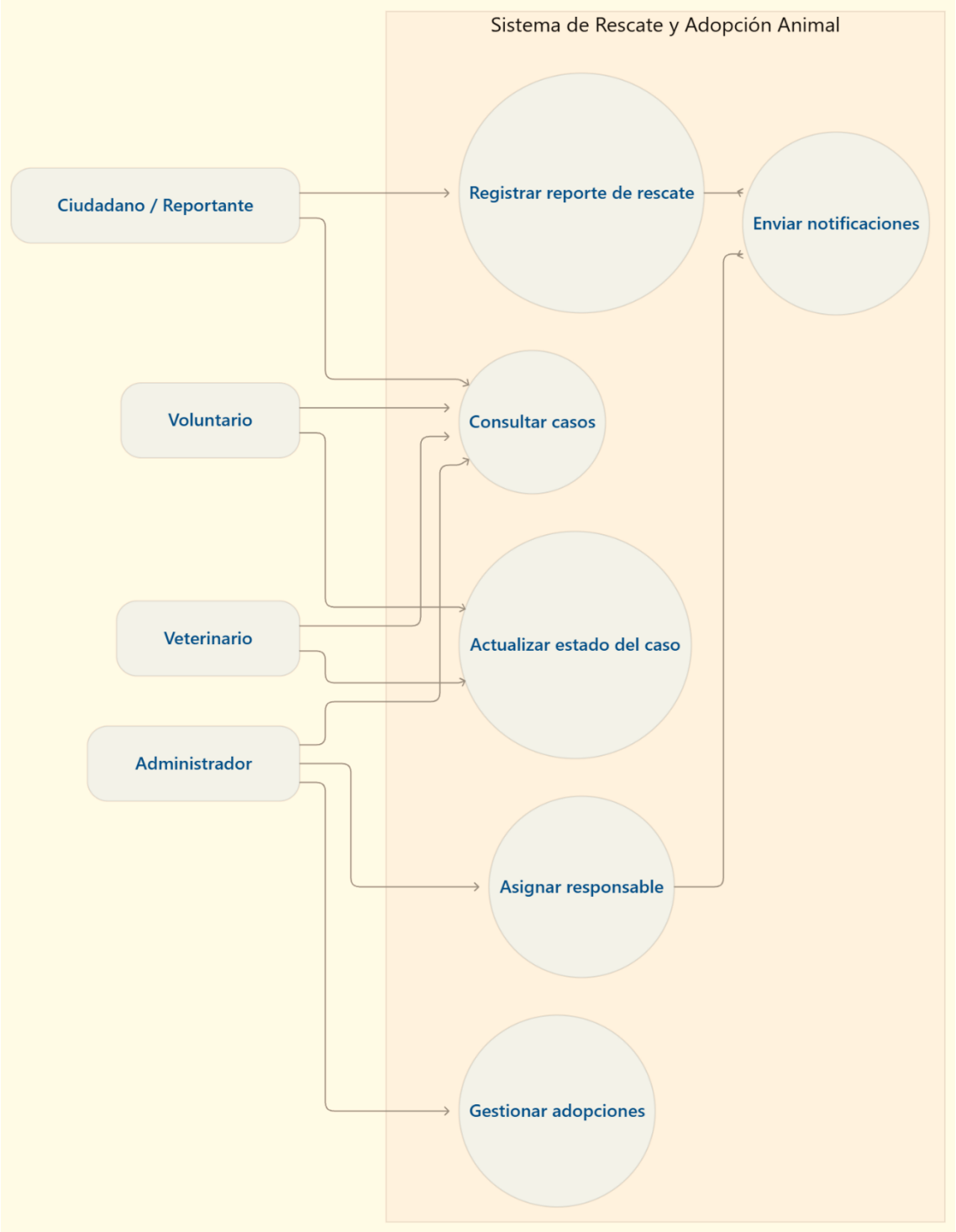


Figura 3: *diseño de diagrama de casos de uso*

Fuente: *Elaboración propia.*

7.4. Diagrama de clases UML

El diagrama de clases presenta la estructura de los principales objetos que conforman el sistema y las relaciones existentes entre ellos.

La clase Animal representa la información general de los animales, mientras que Perro, Gato y FaunaSilvestre representan sus tipos específicos. La clase CasoRescate registra los datos de cada rescate y mantiene la relación con el animal correspondiente y con el usuario responsable.

La clase Usuario representa la información común del personal y se especializa en Voluntario, Veterinario y Administrador. La clase FabricaExpedientes se encarga de crear los expedientes según la especie, mientras que GestorCasos centraliza las operaciones de gestión y Notificador coordina los avisos al personal.

La separación de responsabilidades favorece la mantenibilidad y la reutilización del diseño. También sirve como base para aplicar los principios SOLID, especialmente la responsabilidad única y la inversión de dependencias, siempre que las dependencias se implementen mediante abstracciones adecuadas.

Código:

```
classDiagram
    class CasoRescate {
        +int id
        +String ubicacion
        +String descripcion
        +String estado
        +registrar()
        +actualizarEstado()
    }

    class Animal {
        +int id
        +String nombre
        +String especie
        +String condicion
    }

    class Perro
    class Gato
```

```
class FaunaSilvestre

class Usuario {
    +int id
    +String nombre
    +String correo
}

class Voluntario {
    +asumirCaso()
}

class Veterinario {
    +registrarAtencion()
}

class Administrador {
    +asignarResponsable()
}

class FabricaExpedientes {
    +crearExpediente()
}

class GestorCasos {
    +registrarCaso()
    +consultarCaso()
}

class Notificador {
    +notificar()
}

Animal <|-- Perro
Animal <|-- Gato
Animal <|-- FaunaSilvestre
```

Usuario <|-- Voluntario

Usuario <|-- Veterinario

Usuario <|-- Administrador

CasoRescate "1" --> "1" Animal : registra

CasoRescate "0..*" --> "0..1" Usuario : responsable

FabricaExpedientes ..> Animal : crea

GestorCasos ..> CasoRescate : administra

GestorCasos ..> Notificador : avisa

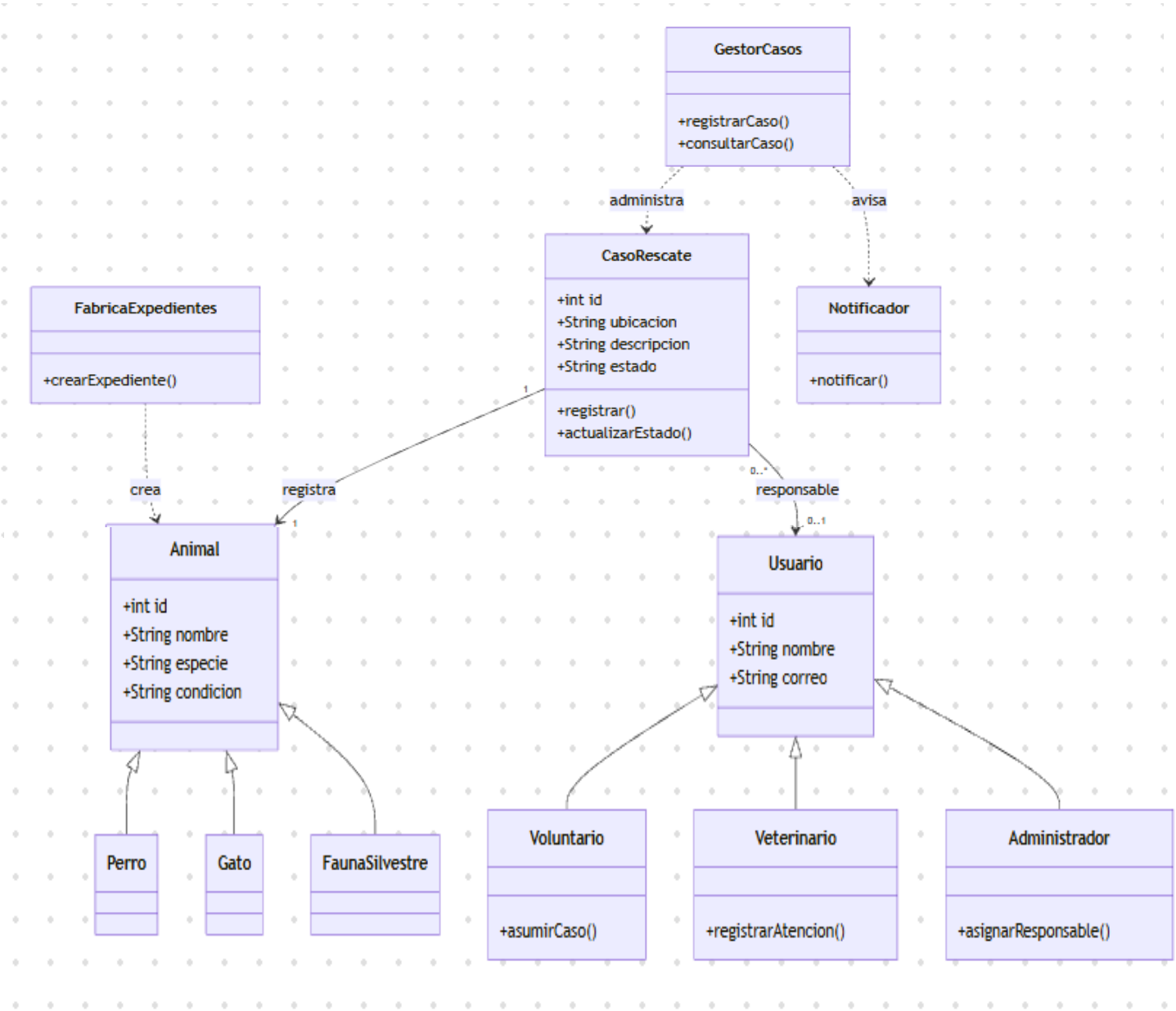


Figura 3: *diagrama de clases UML*

Fuente: *Elaboración propia.*

8. APLICACIÓN DE PATRONES DE DISEÑO

8.1. Patrón Singleton

Propósito: garantizar que exista una única instancia compartida de un componente que deba centralizar una responsabilidad.

Aplicación en el sistema: se propone utilizar Singleton para el GestorCasos, encargado de coordinar las operaciones principales de gestión de los casos de rescate.

Problema que resuelve: evita crear múltiples instancias independientes del gestor cuando se necesita un punto de acceso común a las operaciones del sistema.

Justificación técnica: facilita la centralización del acceso al gestor de casos. Sin embargo, debe utilizarse con cuidado para evitar dependencias globales innecesarias y facilitar las pruebas del sistema.

8.2. Patrón Observer

Propósito: permitir que varios componentes reciban notificaciones cuando ocurre un evento de interés.

Aplicación en el sistema: cuando se registre un nuevo caso de rescate, el componente encargado de las notificaciones informará a los voluntarios o veterinarios correspondientes.

Problema que resuelve: evita que la lógica de registro tenga que comunicarse directamente con cada destinatario de una notificación.

Justificación técnica: separa el registro de los casos del envío de avisos, facilita incorporar nuevos tipos de notificación y permite mantener una estructura más flexible.

8.3. Patrón Factory Method

- Propósito: organizar la creación de objetos mediante un método de fábrica, evitando que el resto del sistema dependa directamente de las clases concretas que necesita crear.
- Aplicación en el sistema: se utilizará para crear expedientes específicos de perros, gatos o fauna silvestre según la especie seleccionada en el registro.
- Problema que resuelve: evita concentrar toda la lógica de creación de los distintos tipos de expedientes en un único bloque de código.
- Justificación técnica: facilita incorporar nuevos tipos de animales y mantener organizado el proceso de creación. Cada tipo de expediente podrá incorporar los datos y comportamientos particulares que necesite.

8.4. Patrón MVC (Modelo-Vista-Controlador)

- Propósito: separar la presentación, el control de las solicitudes y la gestión de los datos del sistema.
- Aplicación en el sistema: el Modelo representa los datos y las reglas del negocio; la Vista presenta los formularios y la información; y el Controlador coordina las solicitudes realizadas por los usuarios.
- Problema que resuelve: evita mezclar en un mismo componente la interfaz gráfica, las reglas del negocio y el acceso a los datos.
- Justificación técnica: mejora la organización del sistema, facilita las modificaciones y permite trabajar en sus componentes de forma más independiente.

8.5. Relación de los patrones con los principios SOLID

El diseño busca aplicar los principios SOLID mediante la separación de responsabilidades, la utilización de componentes especializados y la posibilidad de ampliar las funciones sin modificar innecesariamente las partes existentes.

El principio de responsabilidad única se refleja en la separación entre la gestión de casos, la creación de expedientes y las notificaciones. El principio abierto/cerrado se considera al diseñar mecanismos que permitan añadir nuevos tipos de animales. La inversión de dependencias puede aplicarse mediante interfaces o abstracciones para reducir el acoplamiento entre los componentes.

Estos principios deberán respetarse durante una futura implementación del sistema.

9. Diagrama de secuencia

El diagrama de secuencia representa el proceso de registro de un nuevo caso de rescate. Primero, el ciudadano ingresa los datos en el formulario. La vista envía la información al controlador, que solicita a Factory Method la creación del expediente correspondiente a la especie seleccionada.

Después, el controlador entrega el expediente al GestorCasos para registrar el caso. Una vez confirmado el registro, se genera un evento que permite al componente Observer notificar al personal responsable. Finalmente, la vista informa al ciudadano que el caso ha sido registrado.

Este flujo muestra cómo los patrones de diseño colaboran para organizar el proceso, separar responsabilidades y facilitar la incorporación de nuevas funcionalidades.

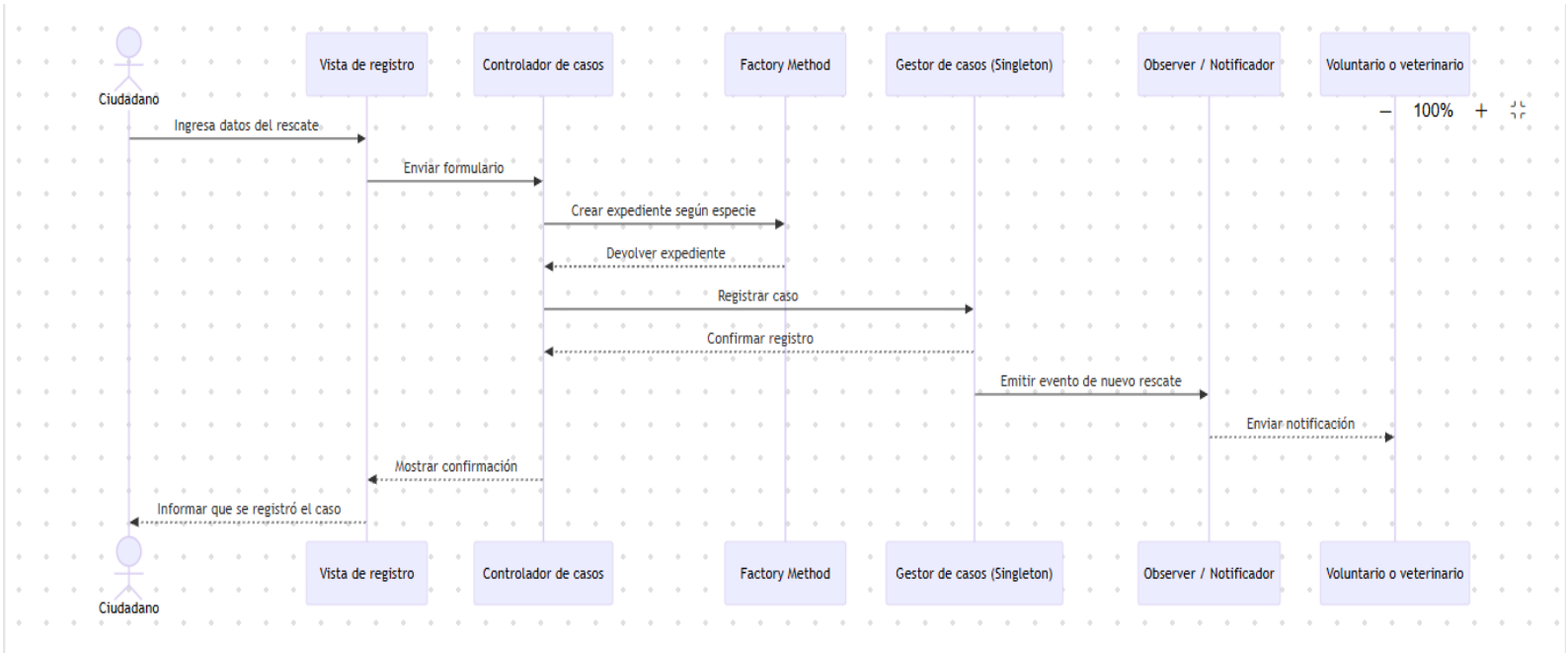


Figura 3: *diagrama de secuencia*

Fuente: *Elaboración propia.*

CONCLUSIONES

- El diseño propuesto permite organizar la información de los animales rescatados, los responsables asignados y los estados de cada caso, facilitando el seguimiento del proceso de rescate y adopción.
- La aplicación de los patrones Singleton, Observer, Factory Method y MVC permite distribuir las responsabilidades del sistema y establecer una estructura que favorece su mantenimiento y ampliación.
- El uso de GitHub y Gitflow permite organizar los avances del proyecto, registrar los cambios realizados y mantener un historial de versiones que facilita la revisión del trabajo.

Bibliografía

Git. (S.F). *Git documentation*. Obtenido de Git documentation: https://git-scm.com/doc

GitHub. (S.f). *GitHub documentation*. Obtenido de GitHub documentation: https://docs.github.com/

Group, O. M. (S.f). *Unified Modeling Language (UML)*. Obtenido de Unified Modeling Language (UML): https://www.omg.org/spec/UML/

Mermaid. (S.f). *Mermaid documentation*. Obtenido de Mermaid documentation: https://mermaid.js.org/

Refactoring.Guru. (S.f). *Design patterns.* Obtenido de Design patterns.: https://refactoring.guru/design-patterns

Regalado, D. (19 de Enero de 2025). *medium*. Obtenido de medium: https://davidregalado255.medium.com/qu%C3%A9-es-gitflow-36dc19b5b8ec